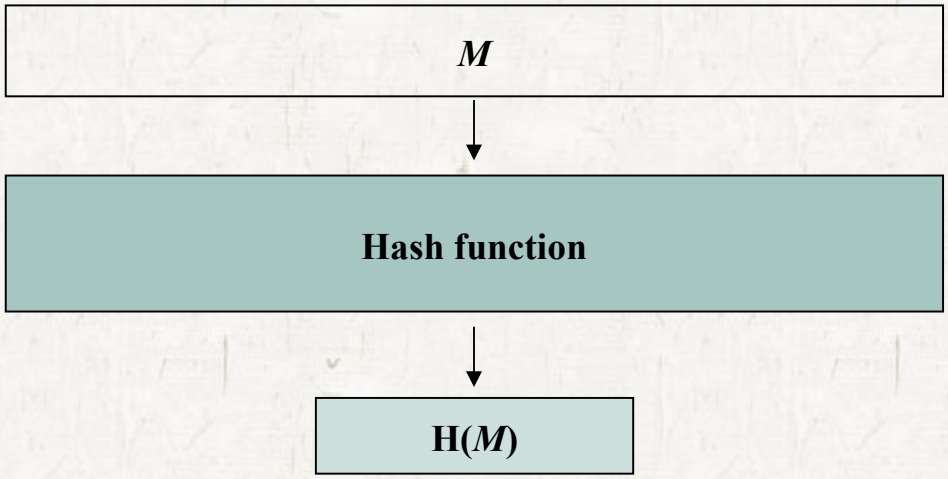
Hash and Mac Algorithms

Contents

- Hash Functions
- Secure Hash Algorithm
- HMAC

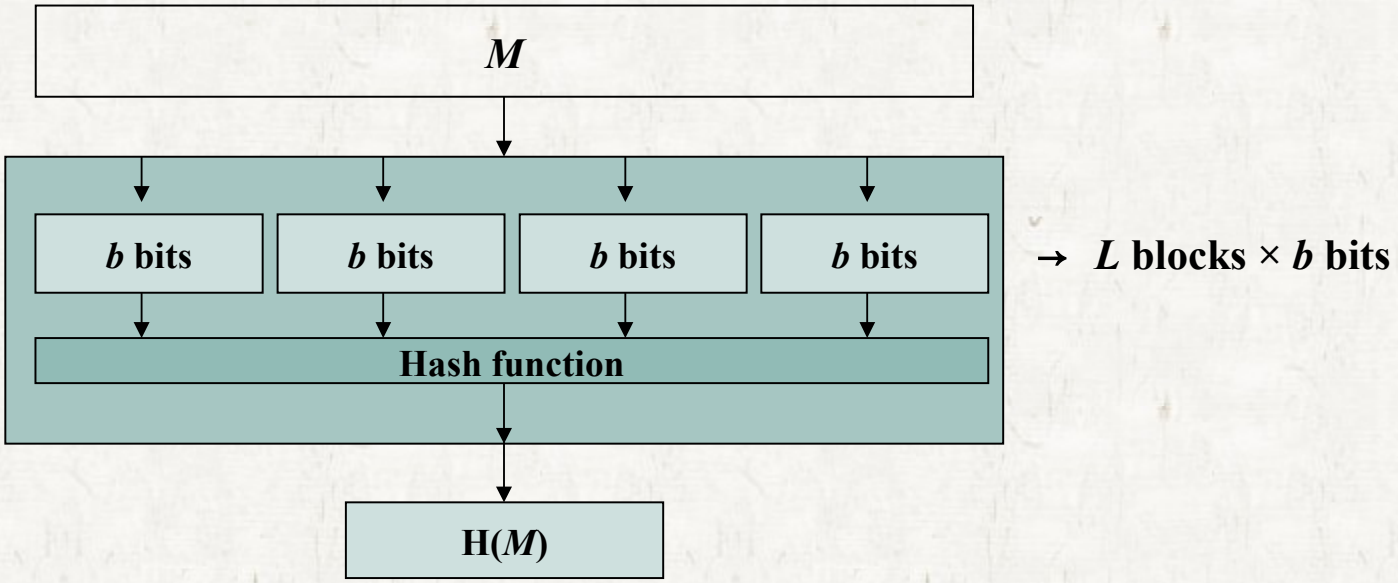
Hash Functions

- **Hash functions**
  - Takes an input message *M*
  - Produces an output hash value, H(*M*), for the message *M*.



Hash Functions

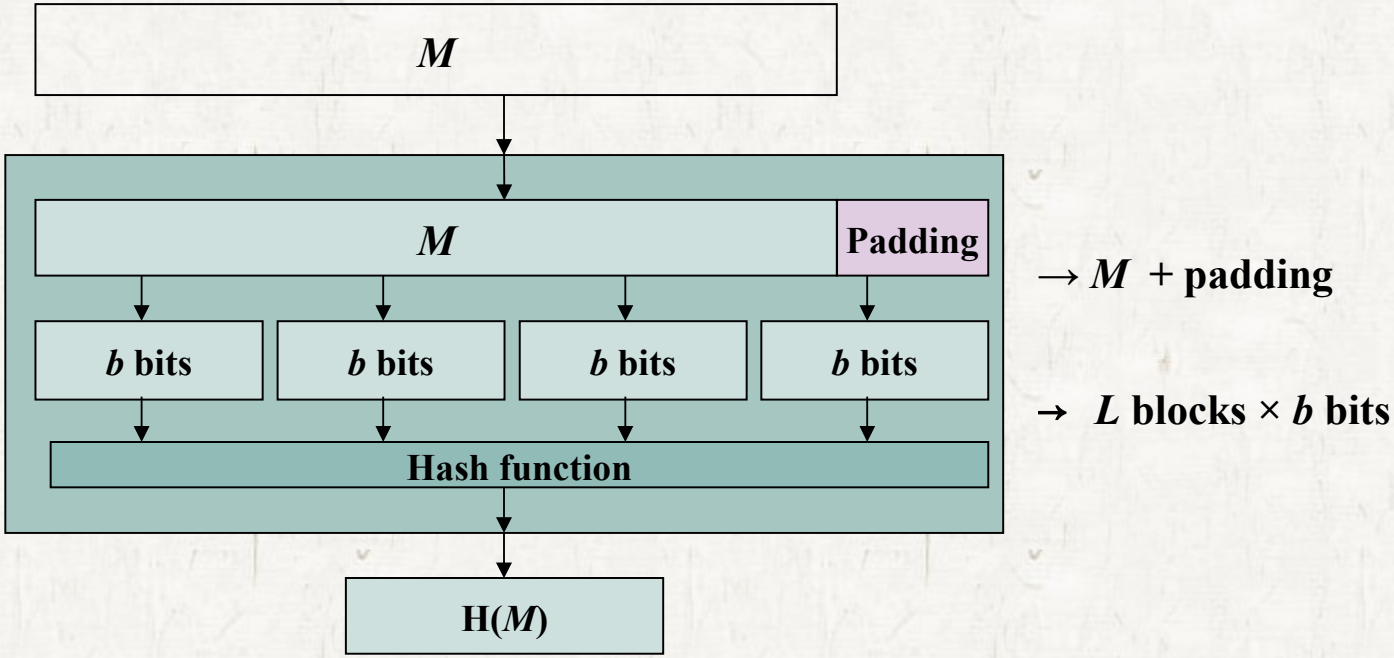
- **Hash functions**



  - partitions it into L fixed-size blocks of b bits each

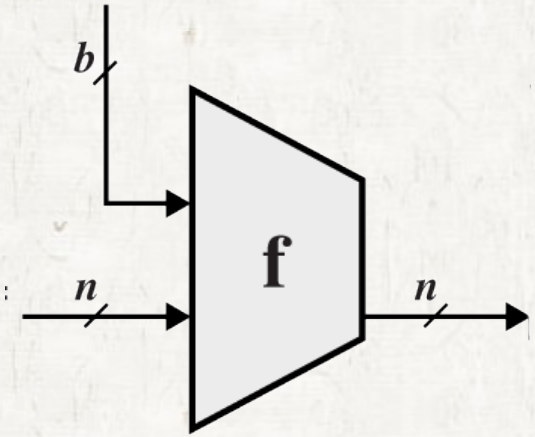
Hash Functions

- If necessary, the final block is padded to b bits
  - Modify the length of M to L blocks × b bits



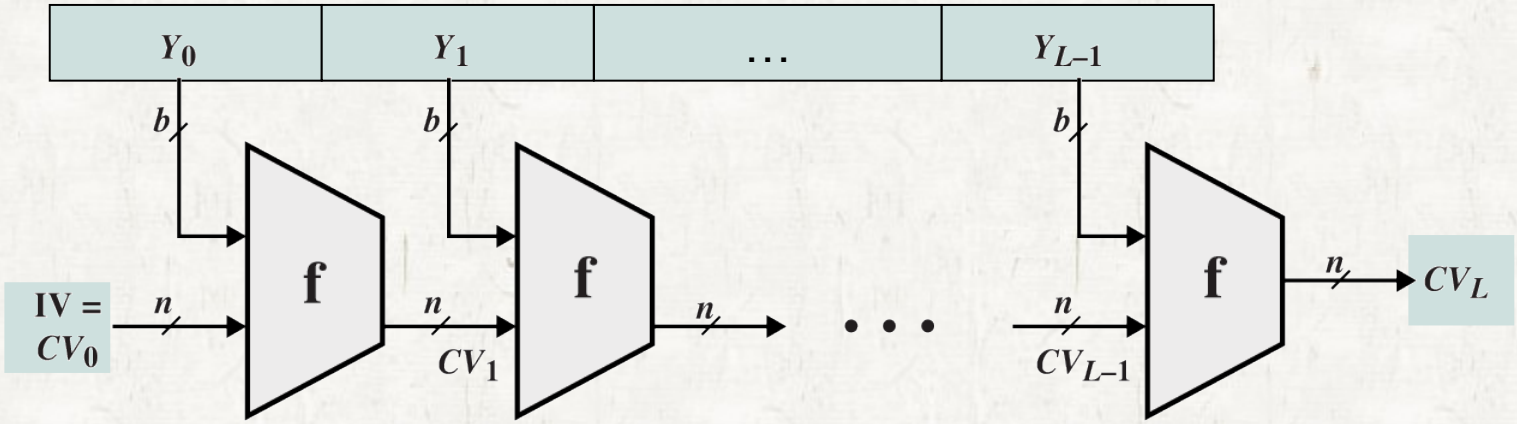
Hash Functions

- **Compression function, f**
  - Hash algorithm involves repeated use of **compression function, f**
    - takes an *n*-bit input **from previous step** and a *b*-bit input **from message.**
    - produces an *n*- bit output.



Hash Functions

- **Hash functions**



IV or CV_0	Initial value for 1st compression
CV_i	Output of the *i*th compression
CV_L	The final hash value, H(*M*)
n	Length of hash code
Y_i	*i*th input block from message *M*
b	Length of input block

Secure Hash Algorithm

- **SHA (Secure Hash Algorithm)**
  - developed by NIST and published as FIPS 180 in 1993
    - NIST, National Institute of Standards and Technology
    - FIPS, a federal information processing standard
  - revised version FIPS 180-1 was issued in 1995
    - referred to as SHA-1 that produces 160 bit hash value.
  - FIPS 180-2 in 2002 defined 3 versions of SHA
    - SHA-256, SHA-384 and SHA-512 for 256, 384 and 512 bits hash.

Secure Hash Algorithm

- SHA-1 is based on the hash function MD4.
- SHA-256, SHA-384, SHA-512
  - have the same underlying structure as SHA-1
  - also use the same types of modular arithmetic and logical binary operation as SHA-1

- Comparison of 4 version of SHA

	SHA-1	SHA-256	SHA-384	SHA-512
Message digest size	160	256	384	512
Message size	$< 2^{64}$	$< 2^{64}$	$< 2^{128}$	$< 2^{128}$
Block size	512	512	1024	1024
Word size	32	32	64	64
Number of steps	80	64	80	80
Security	80	128	192	256

Secure Hash Algorithm

- **SHA-512 Logic**
  - Input : a maximum length of less than $< 2^{128}$ bits
  - Output : a 512-bit message digest

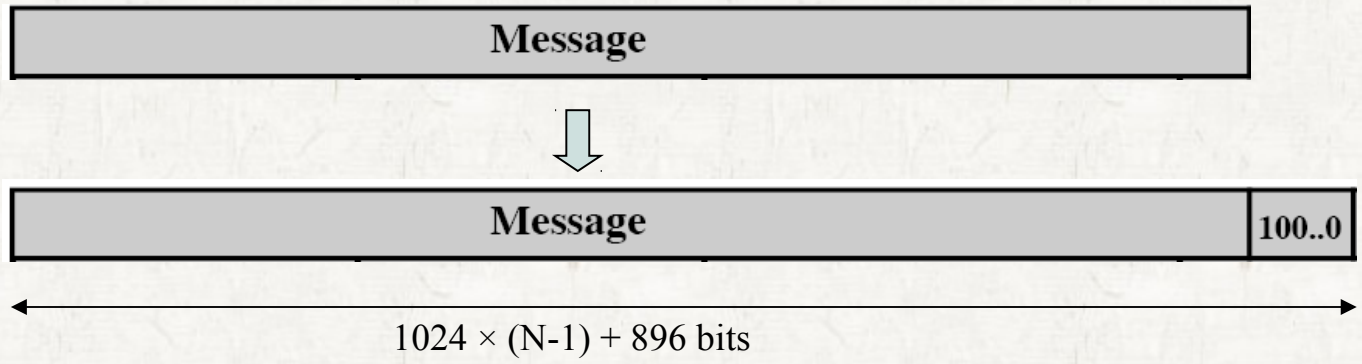
Secure Hash Algorithm

- 5 Steps
  - Step 1: Append padding bits
  - Step 2: Append length
  - Step 3: Initialize hash buffer
  - Step 4: Process message in 1024-bit(128-word) blocks
  - Step 5: Output

Secure Hash Algorithm

1. append padding
2. append length
3. Initialize hash buffer
4. Process message
5. Output

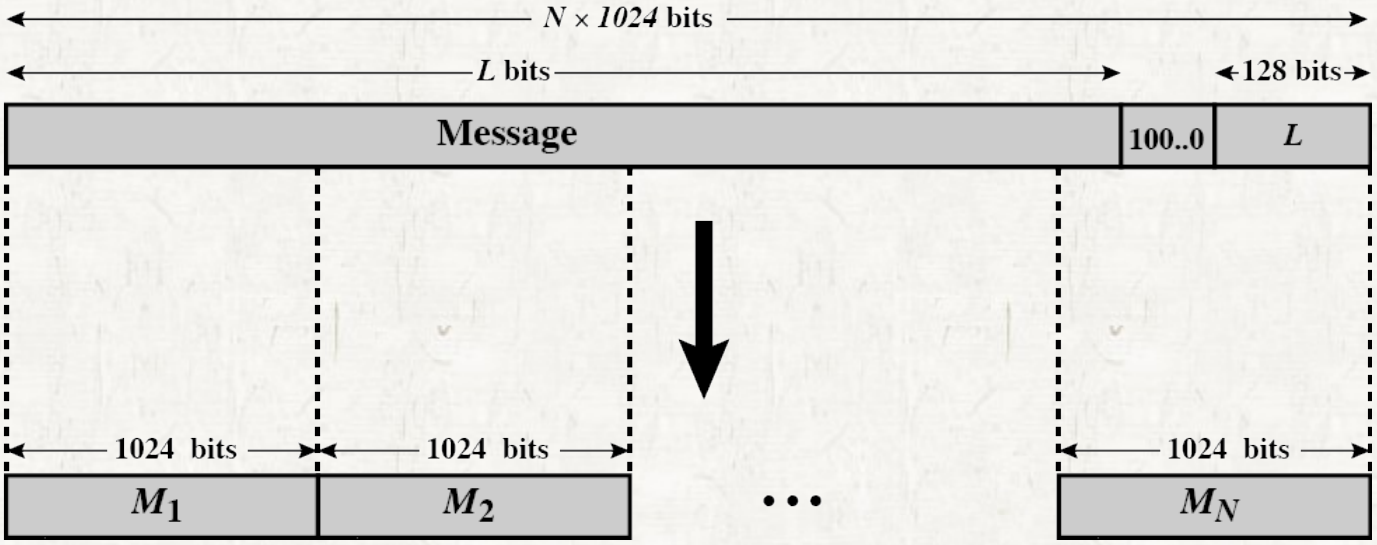
- **Step 1: Append padding bits**
  - The message is padded so that its length is congruent to 896 mod 1024, [length ≡ 896 mod 1024]
  - Padding is always added, even if the length of message is satisfied.
    - If the length of message is 896 bits, padding is 1024 bits, because 1920(=896 + 1024) mod 1024 = 448.
    - thus, 1 ≤ padding bits ≤ 1024
  - The padding consists of a single 1-bit followed by the necessary number of 0-bits, (100…0)



Secure Hash Algorithm

1. append padding
2. append length
3. Initialize hash buffer
4. Process message
5. Output

- **Step 2: Append length**
  - A block of 128 bit is appended to the message
    - contains the length of the original message (before padding)
  - After 2 steps, the length of message is a multiple of 1024
    - The expanded message is a sequence of 1024 bit block M_1, …, M_N



Secure Hash Algorithm

1. append padding
2. append length
3. Initialize hash buffer
4. Process message
5. Output

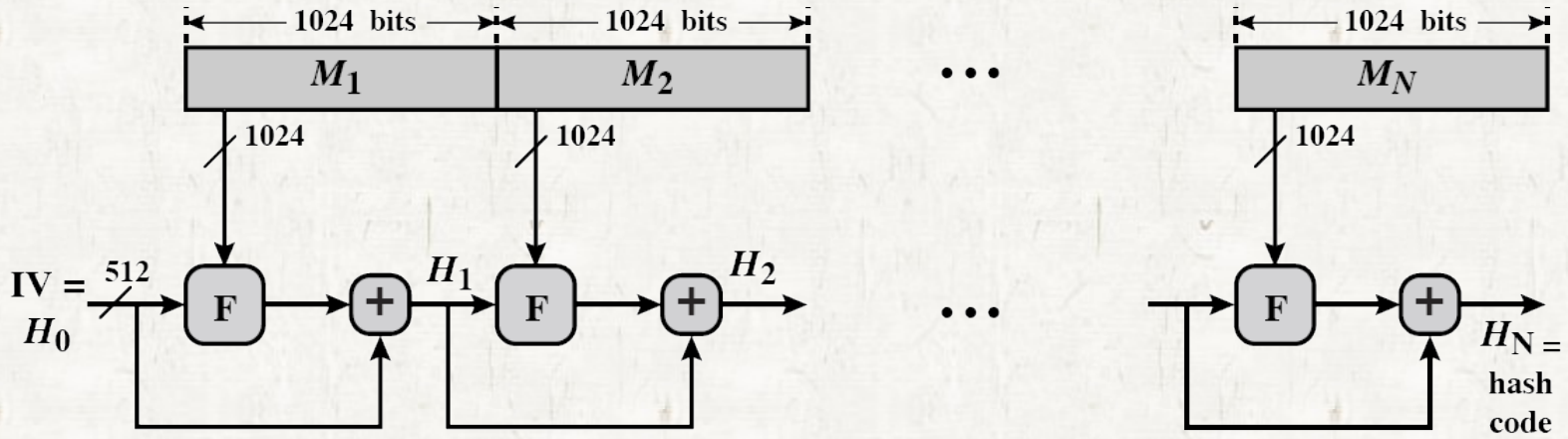
- **Step 3 : Initialize hash buffer**
  - Secure hash algorithm use a 512-bit buffer.
    - holding the intermediate and final result of the hash function.
  - Eight 64-bit registers (a, b, c, d, e, f, g, h) are used.
    - IV(Initial vector) of eight 64-bit registers in hexadecimal value.
    - These words were obtained by taking the first 64bits of the fractional parts of the square roots of the first 80 prime numbers.

a	=	6A09 E667 F3BC C908	e	=	510E 527F ADE6 82D1
b	=	BB67 AE85 84CA A73B	f	=	9B05 688C 2B3E 6C1F
c	=	3C6E F372 FE94 F82B	g	=	1F83 D9AB FB41 BD6B
d	=	A54F F53A 5F1D 36F1	h	=	5BE0 CD19 137E 2179

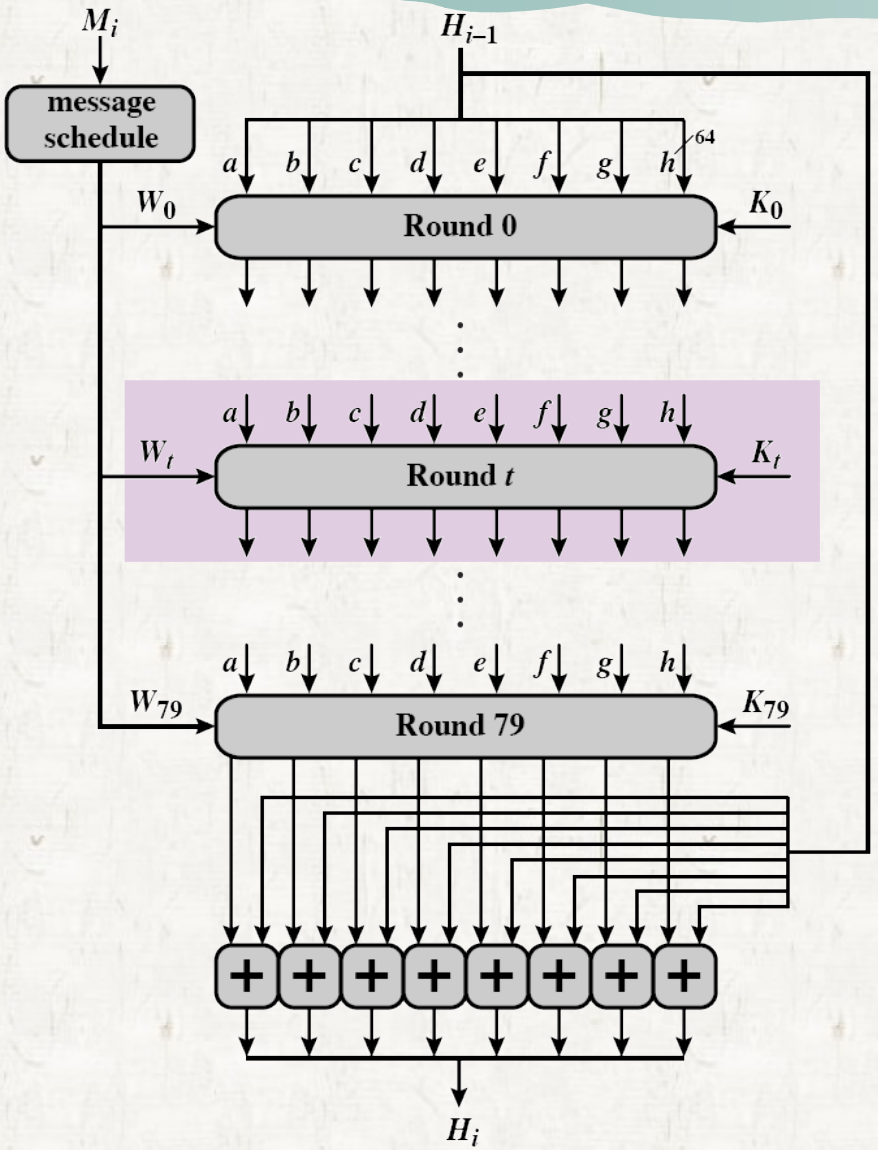
Secure Hash Algorithm

1. append padding
2. append length
3. Initialize MD buffer
4. Process message
5. Output

- **Step 4 : Process message in 1024-bit (128-word) blocks.**
  - The main function of the algorithm is module **F** in the below picture.
    - The module **F** is the compression function.
    - M_i is the ith input block of expanded message.
    - H_i is the intermediate hash result and H_N is the final result.
    - The operation (+) is word-by word addition mod 2^{64}.



Secure Hash Algorithm



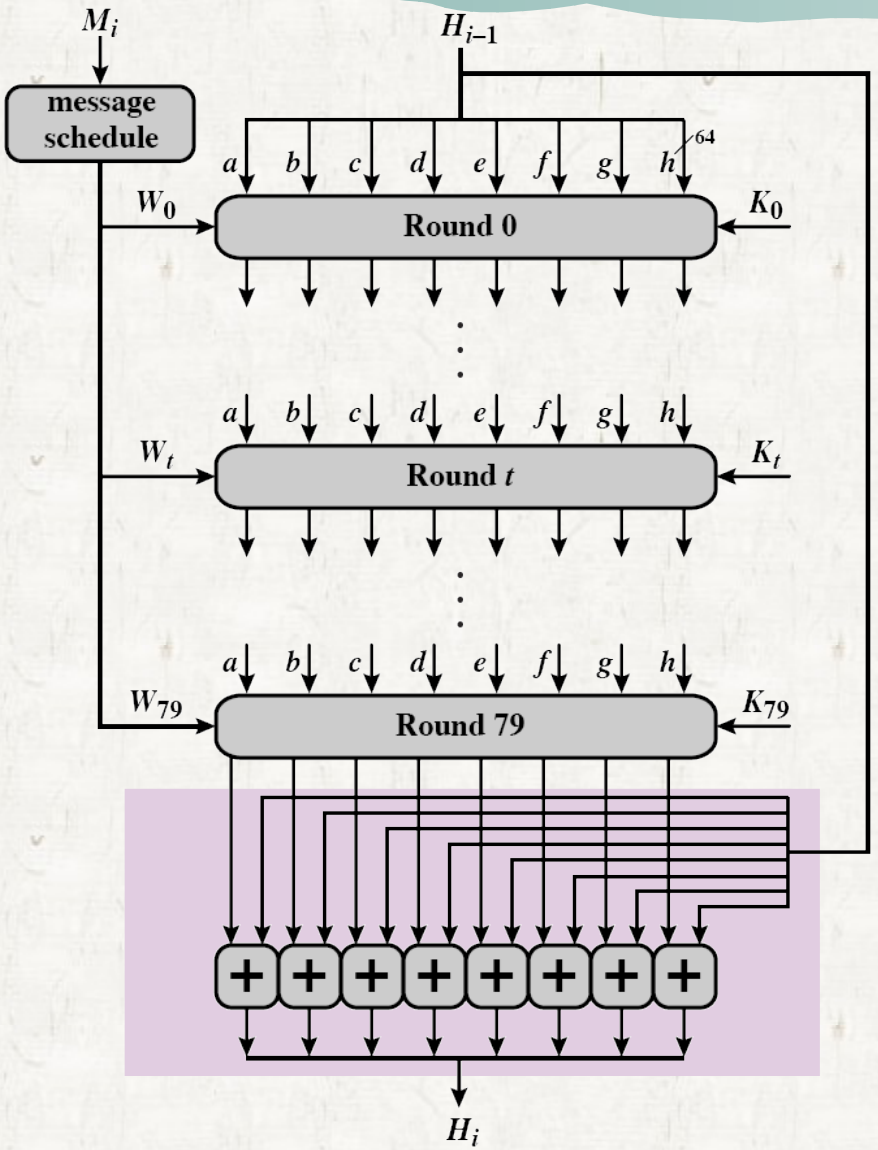
- **The module F consists of 80 rounds for 1 block, M_i**
  - Let t-th round call round t
    - where $0 \leq t \leq 79$
- **Round t takes as an input**
  - the contents of 512bit buffer, abcdefg
  - a 64-bit value, W_t
  - an additive constant, K_t
- **Round t updates**
  - the contents of buffer for the t +1 round

Secure Hash Algorithm

- **W_t, a 64-bit value**
  - A part of message block M_i is used at round t.
  - These values derived from the current 1024-bit block, M_i
  - Detail explain how to generate will be later.

- **K_t**, an additive constant
  - An integer number is added at round t.
  - These words represent the first 64-bits of fractional parts of the cube roots of the first 80 prime number.
  - K_t provides a "randomized" set of 64-bit patterns
    - which eliminate any regularities in the input data.

Secure Hash Algorithm



- **After 80th rounds, the contents of the buffer is added to the input to the first round (H_{i-1}) to produce (H_i).**
  - The addition is done independently
    - for each 8 words with each of the corresponding words in H_{i-1}
  - using addition modulo 2^{64}

Secure Hash Algorithm

- **Step 5 : Output**
  - **After all *N* 1024 bits blocks have been processed, the output form the *N*th stage is the 512-bit message digest.**
  - **Summary of SHA-512**

$$H_0 = \text{IV}$$
$$H_i = \text{SUM}_{64}(H_{i-1}, \text{abcdefghi})$$
$$MD = H_N$$

    - ***N*** **= number of blocks in the expanded message**
    - **SUM_{64} = Addition modulo 2^{64} performed separately on each word of the pair of inputs**

Secure Hash Algorithm

- **SHA-512 round function**
  - Detail at the logic in each of the 80 steps of the processing of on 512-bit block.
  - Each round is defined by the following set of equation :

$$a = T_1 + T_2 \qquad e = d + T_1$$
$$b = a \qquad f = e$$
$$c = b \qquad g = f$$
$$d = c \qquad h = g$$

    - T_1 and T_2 will be shown in the next slide.

Secure Hash Algorithm

$$T_1 = h + Ch(e, f, g) + (\sum_1^{512} e) + W_t + K_t$$

$$T_2 = (\sum_0^{512} a) + Maj(a, b, c)$$

t = step number; $0 \le t \le 79$

$Ch(e, f, g)$ = $(e \text{ AND } f) \oplus (\text{NOT } e \text{ AND } g)$

Maj = $(a \text{ AND } b) \oplus (a \text{ AND } c) \oplus (b \text{ AND } c)$

$(\sum_0^{512} a)$ = $\text{ROTR}^{28}(a) \oplus \text{ROTR}^{34}(a) \oplus \text{ROTR}^{39}(a)$

$(\sum_1^{512} e)$ = $\text{ROTR}^{14}(e) \oplus \text{ROTR}^{18}(e) \oplus \text{ROTR}^{41}(e)$

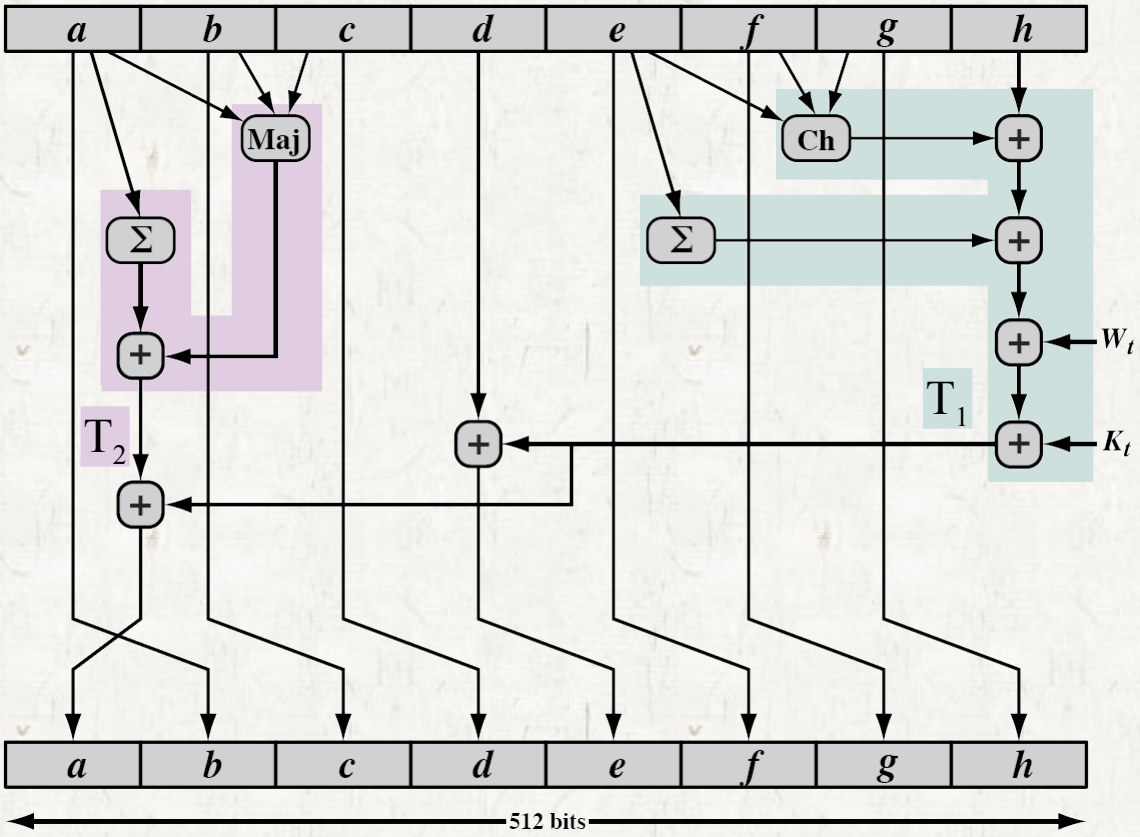
$\text{ROTR}^n(x)$ = circular right shift of the 64bit argument x by n bits

W_t = a 64bit word derived from the current 1024bit input block

K_t = a 64bit additive constant

$+$ = addition modulo 2^{64}

Secure Hash Algorithm



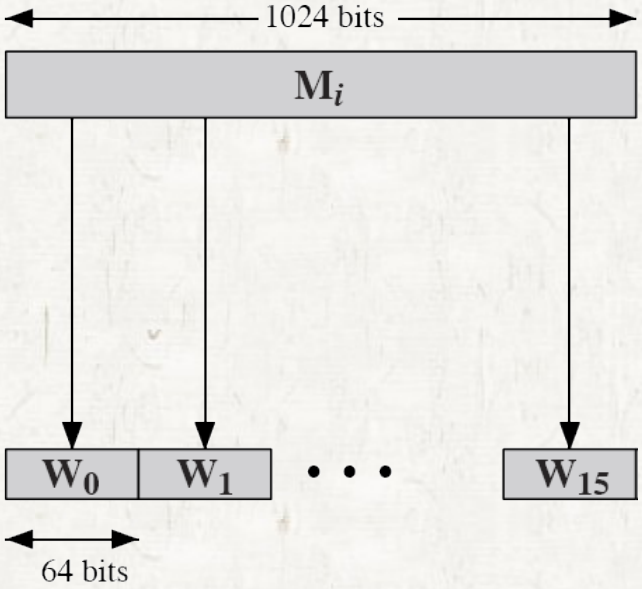
- $a = T_1 + T_2$ $\quad e = d + T_1$
  $b = a$ $\quad f = e$
  $c = b$ $\quad g = f$
  $d = c$ $\quad h = g$

$$T_1 = h + Ch(e,f,g) + (\sum_1^{512} e) + W_t + K_t$$

$$T_2 = (\sum_0^{512} a) + Maj(a,b,c)$$

Secure Hash Algorithm

- **W_t, a 64-bit value**
  - W_t are derived from the 1024-bit message.
  - The first 16 values of W_t are taken directly from the 16 words of the current block.



Secure Hash Algorithm

- The remaining values are defined as follows.

$$W_t = W_{t-16} + \sigma_0^{512}(W_{t-15}) + W_{t-7} + \sigma_1^{512}(W_{t-2})$$

where

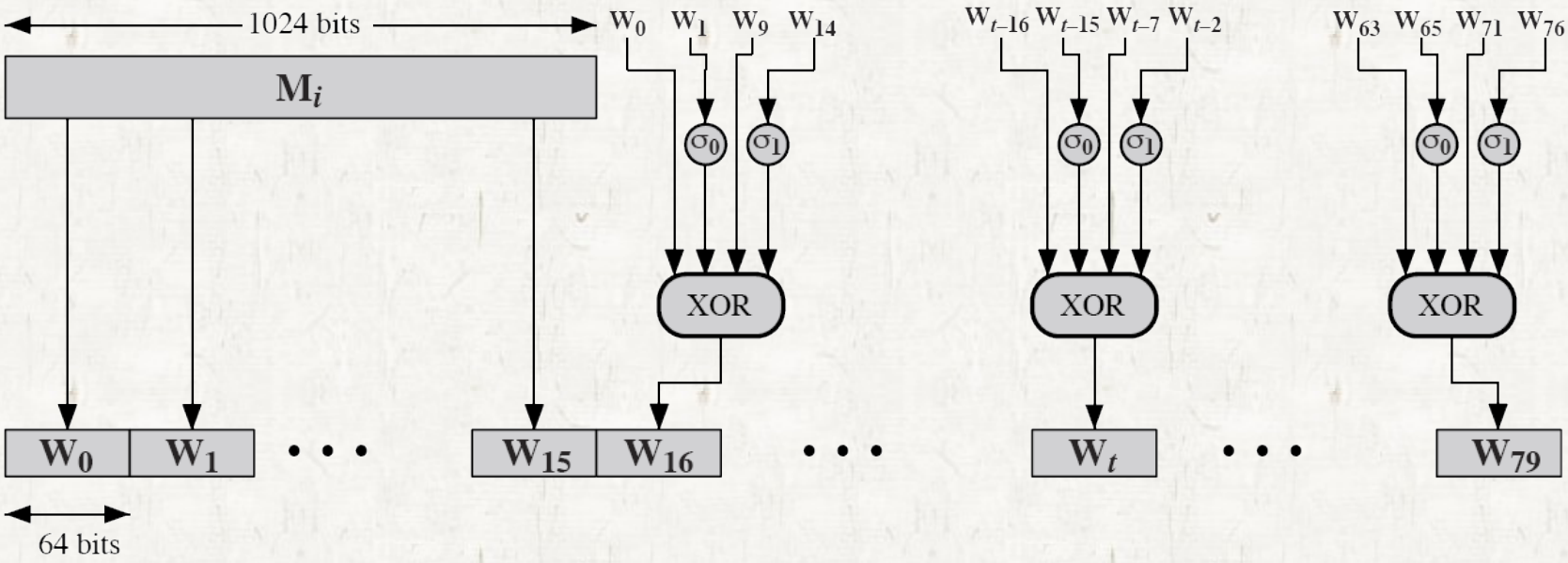
$$\sigma_0^{512}(x) = \mathrm{ROTR}^1(x) \oplus \mathrm{ROTR}^8(x) \oplus \mathrm{SHR}^7(x)$$

$$\sigma_1^{512}(x) = \mathrm{ROTR}^{19}(x) \oplus \mathrm{ROTR}^{61}(x) \oplus \mathrm{SHR}^6(x)$$

$\mathrm{SHR}^n(x) =$ left shift of the 64-bit argument x by n bits with padding *by zeroes on the right*

Secure Hash Algorithm

- Creation of W_t



$$W_t = W_{t-16} + \sigma_0^{512}(W_{t-15}) + W_{t-7} + \sigma_1^{512}(W_{t-2})$$

$$Ex)\ W_{16} = W_0 + \sigma_0^{512}(W_1) + W_{11} + \sigma_1^{512}(W_{14})$$

HMAC

- **MAC (A message authentication code)**
  - defined FIPS SUB 113
  - The most common approach to construct a MAC
  - Recently, there has been increased interest in developing a MAC.

  - The motivation
    1. cryptographic hash function, MD5 and SHA-1, generally execute faster in software than symmetric block cipher such as DES.
    2. Library code for cryptographic hash functions is widely available.

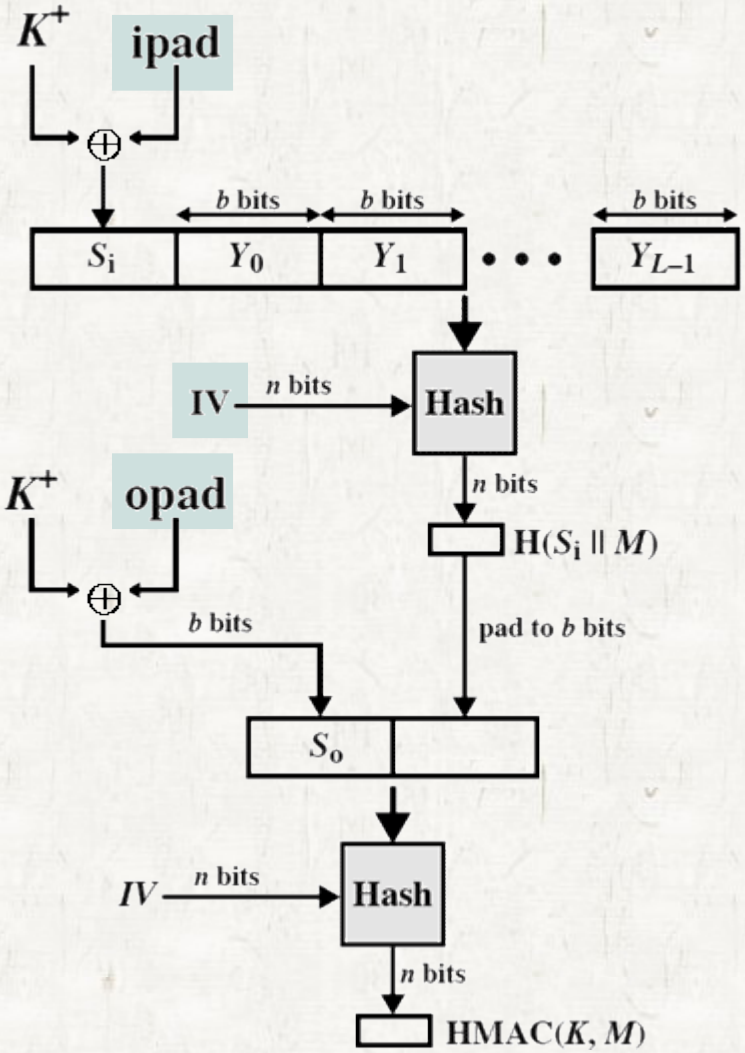
HMAC

- A hash function such as SHA
  - not designed for use as a MAC
  - cannot be used directly for that purpose because it does not rely on the secret key.
- There have been a number of proposals
  - for the incorporation of a secret key into an existing hash algorithm
- HMAC[BELL96a] is most supported.
  - issued RFC 2104 and as a NIST(FIPS 198).
  - as the mandatory-to-implement MAC for IP security
  - used in other Internet protocol such as SSL.

HMAC

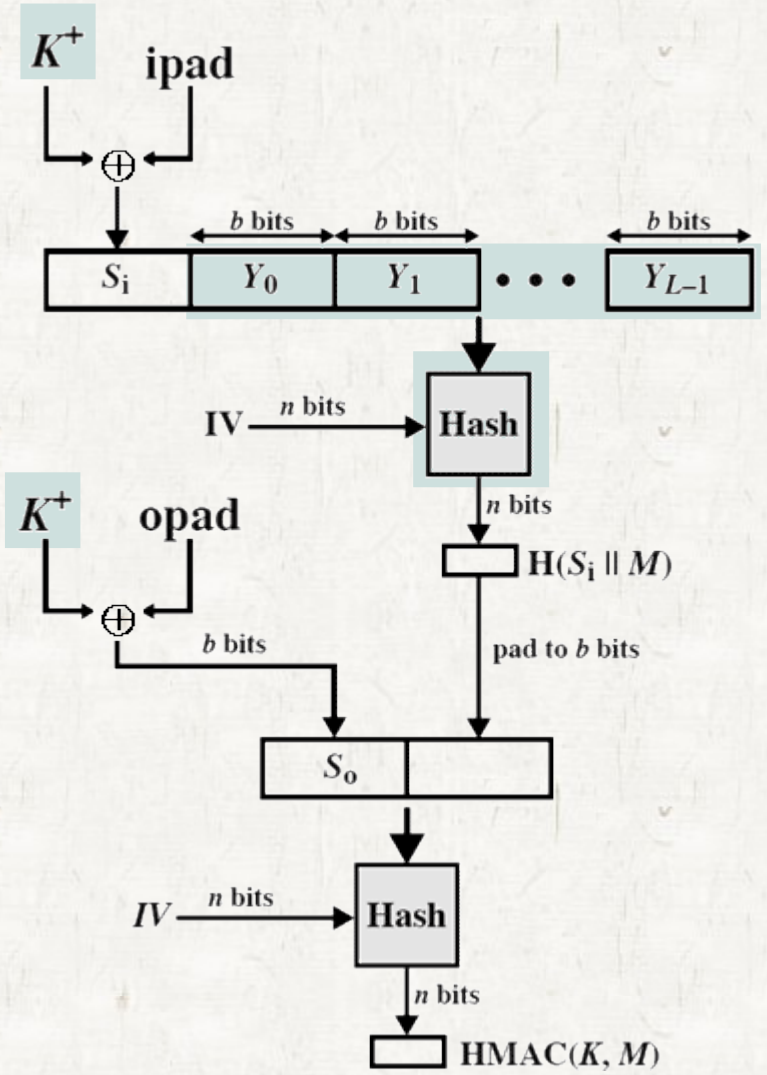
- **HMAC Design Objectives on RFC 2104 list**
  - To use, without modification, available hash functions. In particular, hash functions that perform well in software and code is freely and widely available.
  - To allow for easy replaceability of the embedded hash function in case faster or more secure hash function are found or required.
  - To preserve the original performance of the hash function without incurring a significant degradation.
  - To use and handle key in a simple way.
  - To have a well understood cryptographic analysis of the strength of the authentication mechanism based on reasonable assumption about the embedded hash function.

HMAC



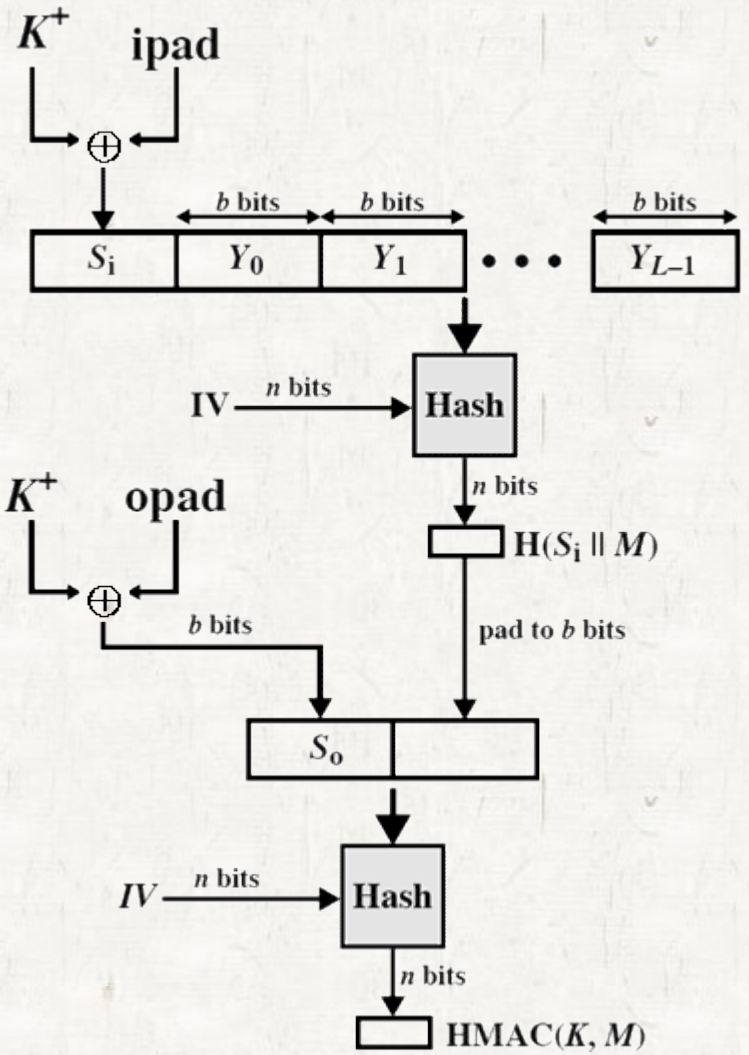
- **HMAC structure**
  - IV = initial value input to hash function
  - M = message input to HMAC
  - K = secret key recommended length is $\geq n$;
    - if key length is greater than b; the key is input to the hash function to produce an n-bit key.
  - ipad = 00110110 repeated $b/8$ times
  - opad = 01011100 repeated $b/8$ times

HMAC



- **HMAC structure**
  - Hash = embedded hash function (MD5, SHA-1, RIPEMD-160)
  - Y_i = ith block of M, $0 \leq i \leq (L\text{-}1)$
  - K^+ = K padded with 0 on left so that the result is b bits in length
  - L = number of blocks in M
  - b = number of bits in a block
  - n = length of hash code produced by H

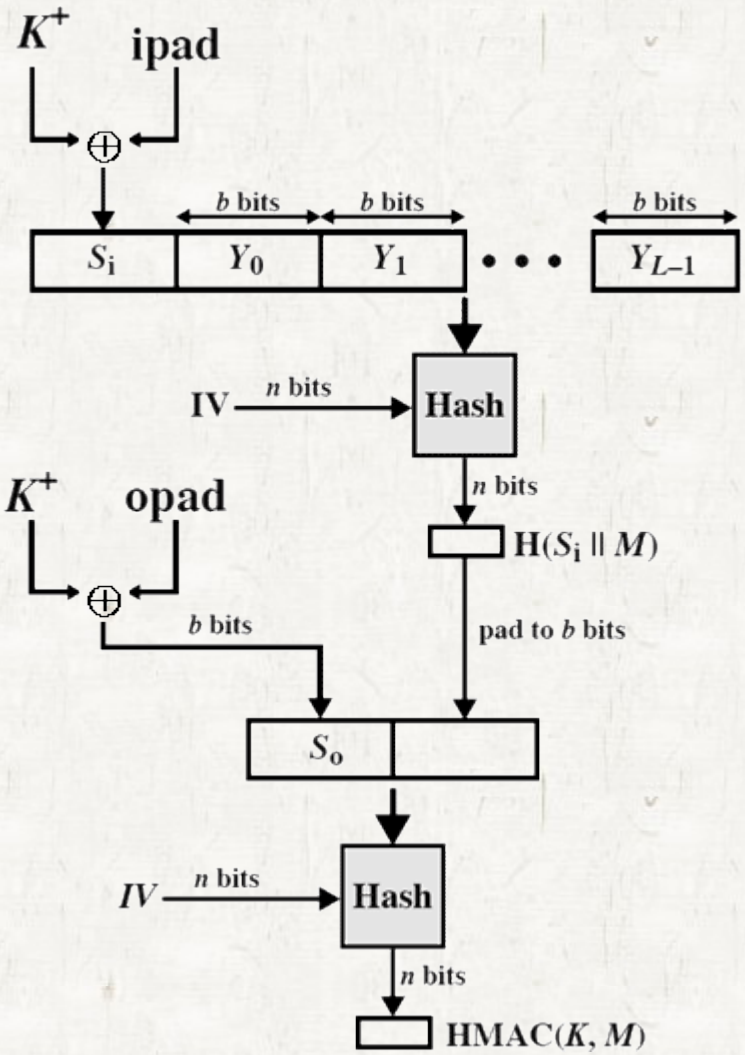
HMAC



HMAC Algorithm

1. **Append zero to the left end of *K* to create a *b*-bit string K^+**
   - if *K* is of length 160 bits and $b = 512$, *K* will be appended with 44 zero bytes 0×00.
   - K^+ = *K* padded with 0 on left so that the result is *b* bits in length
2. **XOR K^+ with ipad to produce the *b*-bit block S_i**
   - ipad = 00110110
3. **Append *M* to S_i**

HMAC



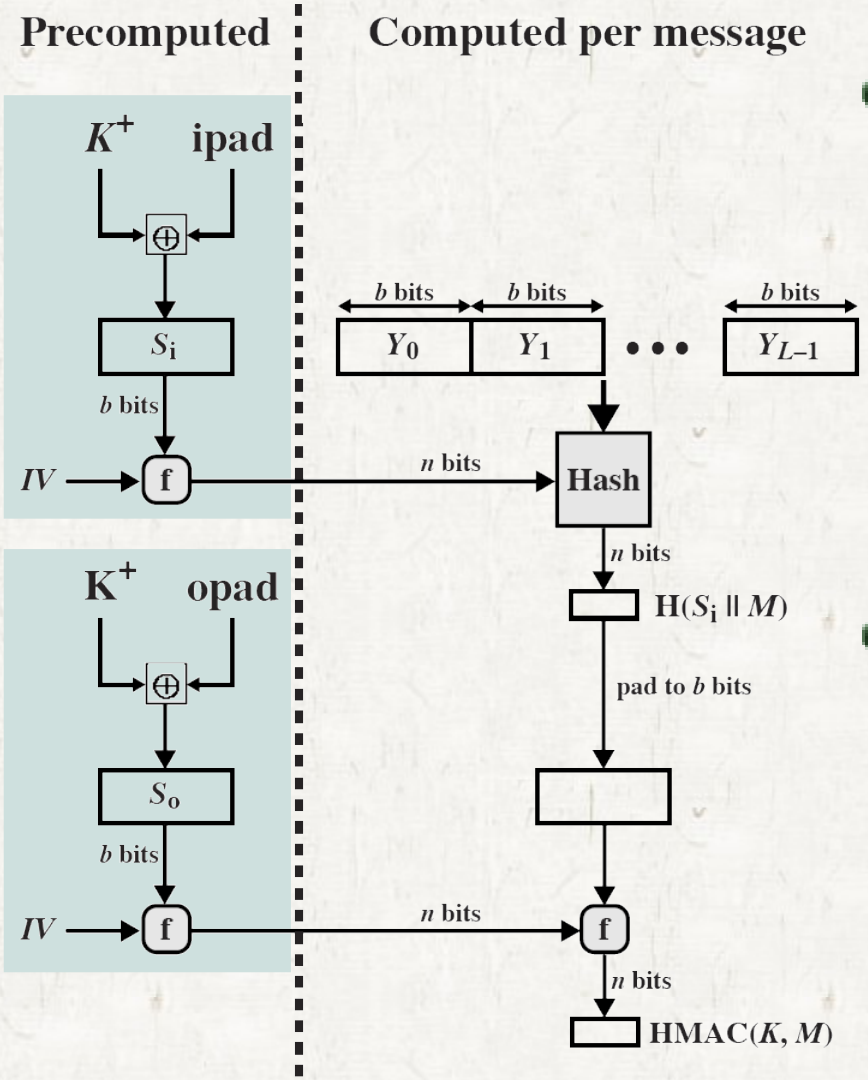
4. **Apply H to the stream generated in step 3.**

5. **XOR K^+ with opad to produce the *b*-bit block S_0.**
   - opad = 01011100

4. **Append the hash result from step 4 to S_0.**

5. **Apply H to the stream generated in step 6 and output result.**

HMAC



- **HMAC should execute in approximately the same time as the embedded hash function**
  - for a long message.
  - HMAC adds 3 executions of the hash compression function.
- **A more efficient implement is possible by precomputing**

$$f(\mathrm{IV}, (K^+ \oplus \mathrm{ipad}))$$

$$f(\mathrm{IV}, (K^+ \oplus \mathrm{opad}))$$

HMAC

$$f(IV, (K^+ \oplus ipad))$$

$$f(IV, (K^+ \oplus opad))$$

- These quantities only need to computed initially and every time the key exchange.
- The precomputed quantities substitute for the initial value.

- Only one additional instance of the compression function is added to the processing.

HMAC

- **Security of HMAC**
  - The security of any MAC function based on an embedded hash function depends in some way on the cryptographic strength of the underlying hash function.
  - The appeal of HMAC is that its designers have been able to prove an exact relationship between the strength of the embedded hash function and the strength of HMAC

HMAC

- The security of HMAC is expressed in terms of the probability of successful forgery with
  - a given amount of time spent by the forger
  - a given number of message-MAC pairs created with the same key.

- For a given level of effort (time, message-MAC pairs) on messages generated by a legitimate user and seen by the attacker, the probability successful attack on HMAC is equivalent to one of following attacks.

HMAC

- **The probability successful attack on HMAC**

1. The attacker is able to compute an output of the compression function even with an IV that is random, secret, and unknown to the attacker.

2. The attacker finds collisions in the hash function even when IV is random and secret.

HMAC

- In the 1st attack, compression function as equivalent to the hash function.
  - For this attack, the IV of the hash function is replaced by a secret, random value of *n* bits.
  - An attack requires either
    - A brute-force attack on the key, a level of effort on the order of 2^n
    - A birthday attack, a special case of 2nd attack.

HMAC

- In the 2nd attack, the attack is looking for 2 messages *M* and *M*' that produce H(*M*)=H(*M*')
  - A birthday attack requires a level of effort of $2^{n/2}$ for a hash length of *n*
  - MD5, 2^{64}, looks feasible in today, so MD5 is unsuitable for HMAC?
  - The answer is no.
    - To attack MD5, attackers know the hash algorithm and IV, so they can generate the hash code for any message
    - In HMAC, attackers don't know *K,* so they can't generate the hash code.
    - So, to attack HMAC, attackers must observe a sequence of messages.
    - For a hash code of 128 bits, this requires 2^{64} observed blocks with using the same key.
    - On a 1-Gbps, it takes 150,000 years to get a satisfied stream.
    - Thus, if speed is concern, MD5 is fully acceptable to use rather than SHA-1 as the embedded hash function for HMAC.